

WHY STACKS?

Arrays give random access, but Stacks enforce **LIFO (Last-In, First-Out) order**. This is essential for matching nested structures, expression evaluation, and backtracking memory!

THE ARRAYDEQUE AHA

Never use `java.util.Stack` in Java! It inherits from `Vector` and uses synchronized methods (slow). Always use `Deque stack = new ArrayDeque<>()`!

1 3 CORE VARIANTS

- **Matching / Pair Cancellation:** Valid parentheses, string reduction, asteroid collisions.
- **Monotonic Stack:** Stack of strictly increasing or decreasing elements to find next greater / previous smaller element in $O(N)$!
- **Expression Evaluation:** Reverse Polish Notation (RPN), infix to postfix operators.

2 VISUALIZING MONOTONIC STACK

Decreasing Stack (Next Greater Element):

Array: [73, 74, 75, 71, 69, 72]
Push 73 → Stack: [73]
Element 74 > 73 → Pop 73! Next Greater of 73 is 74.
Push 74 → Stack: [74]
Element 75 > 74 → Pop 74! Next Greater of 74 is 75.
Push 75, 71, 69 → Stack: [75, 71, 69]

PREREQUISITES & COMPLEXITY REASONING

Operation	ArrayDeque Complexity	Deprecated Stack
Push (top insert)	$O(1)$ amortized	$O(1)$ synchronized
Pop (top remove)	$O(1)$	$O(1)$ synchronized
Peek (top view)	$O(1)$	$O(1)$ synchronized
Monotonic Scan	$O(N)$ total	$O(N)$ total

CLUES THAT SCREAM STACK

1. "Valid parentheses / matching brackets" -> Character Stack
2. "Next greater element" / "Daily temperatures" -> Monotonic Decreasing Stack
3. "Largest rectangle in histogram" -> Monotonic Increasing Stack
4. "Evaluate arithmetic expression / RPN" -> Operand Stack

PATTERN
1

PARENTHESES & BRACKET
MATCHING

e.g. Valid Parentheses (LC 20), Backspace String Compare (LC 844)

WHEN TO USE

Push opening brackets `(`, `[`, `{`. When closing bracket encountered, verify top of stack matches and pop!

TEMPLATE — LC 20

```
public boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(') stack.push('(');
        else if (c == '{') stack.push('{');
        else if (c == '[') stack.push('[');
        else if (stack.isEmpty() || stack.pop() != c) return false;
    }
    return stack.isEmpty();
}
```

PATTERN 2 PAIR CANCELLATION & SIMULATION e.g. Asteroid Collision (LC 735), Remove All Adjacent Duplicates**WHEN TO USE**

Simulate collision/cancellation between incoming elements and previous active elements at top of stack.

TEMPLATE — LC 735

```
public int[] asteroidCollision(int[] asteroids) {
    Deque<Integer> stack = new ArrayDeque<>();
    for (int a : asteroids) {
        while (!stack.isEmpty() && a < 0 && stack.peek() > 0) {
            int diff = a + stack.peek();
            if (diff < 0) stack.pop(); // Incoming destroys top
            else if (diff == 0) { stack.pop(); a = 0; } // Both destroyed
            else a = 0; // Incoming destroyed
        }
        if (a != 0) stack.push(a);
    }
    int[] res = new int[stack.size()];
    for (int i = res.length - 1; i >= 0; i--) res[i] = stack.pop();
    return res;
}
```

PATTERN
5

LARGEST RECTANGLE IN HISTOGRAM (Monotonic Increasing Stack)

e.g. Largest Rectangle in Histogram (LC 84),
Maximal Rectangle

MONOTONIC INCREASING STACK
MECHANICS

When `heights[i]` is smaller than stack top, the popped height's right boundary is `i`, and left boundary is current stack top!

TEMPLATE — LC 84

```
public int largestRectangleArea(int[] heights) {
    int n = heights.length, maxArea = 0;
    Deque<Integer> stack = new ArrayDeque<>();
    for (int i = 0; i <= n; i++) {
        int h = (i == n) ? 0 : heights[i];
        while (!stack.isEmpty() && h < heights[stack.peek()]) {
            int height = heights[stack.pop()];
            int width = stack.isEmpty() ? i : i - stack.peek() - 1;
            maxArea = Math.max(maxArea, height * width);
        }
        stack.push(i);
    }
    return maxArea;
}
```

PATTERN
6

EVALUATE REVERSE POLISH NOTATION (Postfix Operators)

e.g. Evaluate Reverse Polish Notation (LC 150)

WHEN TO USE

Push numbers. When operator (+, -, *, /) encountered, pop second operand `b`, pop first operand `a`, evaluate `a op b`, push result!

TEMPLATE — LC 150

```
public int evalRPN(String[] tokens) {
    Deque<Integer> stack = new ArrayDeque<>();
    for (String t : tokens) {
        if (t.equals("+")) stack.push(stack.pop() + stack.pop());
        else if (t.equals("-")) { int b = stack.pop(), a = stack.pop(); stack.push(a - b); }
        else if (t.equals("*")) stack.push(stack.pop() * stack.pop());
        else if (t.equals("/")) { int b = stack.pop(), a = stack.pop(); stack.push(a / b); }
        else stack.push(Integer.parseInt(t));
    }
    return stack.pop();
}
```

COMPLETE STACK PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
20	Valid Parentheses	Parenthesis Match	O(N)	O(N)	E
155	Min Stack	Dual Stack Minimum	O(1) all ops	O(N)	M
150	Evaluate RPN	Operand Stack	O(N)	O(N)	M
739	Daily Temperatures	Monotonic Decreasing	O(N)	O(N)	M
853	Car Fleet	Time Stack	O(N log N)	O(N)	M
735	Asteroid Collision	Collision Simulation	O(N)	O(N)	M
394	Decode String	Nested Dual Stack	O(N)	O(N)	M
84	Largest Rectangle in Histogram	Monotonic Increasing	O(N)	O(N)	H
22	Generate Parentheses	Backtracking Stack	O(4^N / sqrt(N))	O(N)	M

LC 20 · Valid Parentheses

EASY

Pattern: Parenthesis Match

Key Insight: Push expected closing character.

```
public boolean isValid(String s) {
    Deque<Character> st = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(') st.push(')');
        else if (c == '{') st.push('}');
        else if (c == '[') st.push(']');
        else if (st.isEmpty() || st.pop() != c) return false;
    }
    return st.isEmpty();
}
```

LC 155 · Min Stack

MEDIUM

Pattern: Dual Stack
Key Insight: Auxiliary stack tracks minimum value.

```
class MinStack {
    Deque<Integer> st = new ArrayDeque<>(), minSt = new ArrayDeque<>();
    public void push(int val) {
        st.push(val);
        if (minSt.isEmpty() || val <= minSt.peek()) minSt.push(val);
    }
    public void pop() { if (st.pop().equals(minSt.peek())) minSt.pop(); }
    public int top() { return st.peek(); }
    public int getMin() { return minSt.peek(); }
}
```

LC 150 · Evaluate RPN

MEDIUM

Pattern: RPN Operand Stack

Key Insight: Pop `b` then `a`, apply `a op b`.

```
public int evalRPN(String[] tokens) {
    Deque<Integer> st = new ArrayDeque<>();
    for (String t : tokens) {
        if (t.equals("+")) st.push(st.pop() + st.pop());
        else if (t.equals("-")) { int b=st.pop(), a=st.pop(); st.push(a-b); }
        else if (t.equals("*")) st.push(st.pop() * st.pop());
        else if (t.equals("/")) { int b=st.pop(), a=st.pop(); st.push(a/b); }
        else st.push(Integer.parseInt(t));
    }
    return st.pop();
}
```

LC 739 · Daily Temperatures

MEDIUM

Pattern: Monotonic Decreasing**Key Insight:** Stack stores indices. Pop when warmer!

```
public int[] dailyTemperatures(int[] T) {
    int[] res = new int[T.length];
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < T.length; i++) {
        while (!st.isEmpty() && T[i] > T[st.peek()]) {
            int prev = st.pop(); res[prev] = i - prev;
        }
        st.push(i);
    }
    return res;
}
```

LC 853 · Car Fleet

MEDIUM

Pattern: Time Stack**Key Insight:** Sort by position desc. Time $\leq$ top merges.

```
public int carFleet(int target, int[] pos, int[] speed) {
    int n = pos.length; double[][] cars = new double[n][2];
    for (int i = 0; i < n; i++) cars[i] = new double[]{pos[i], (double)(target - pos[i]) / speed[i]};
    Arrays.sort(cars, (a,b)->Double.compare(b[0], a[0]));
    Deque<Double> st = new ArrayDeque<>();
    for (double[] c : cars) {
        if (!st.isEmpty() && c[1] <= st.peek()) continue;
        st.push(c[1]);
    }
    return st.size();
}
```

LC 735 · Asteroid Collision

MEDIUM

Pattern: Collision Simulation**Key Insight:** Destroy smaller top when incoming is negative.

```
public int[] asteroidCollision(int[] ast) {
    Deque<Integer> st = new ArrayDeque<>();
    for (int a : ast) {
        while (!st.isEmpty() && a < 0 && st.peek() > 0) {
            int diff = a + st.peek();
            if (diff < 0) st.pop();
            else if (diff == 0) { st.pop(); a = 0; }
            else a = 0;
        }
        if (a != 0) st.push(a);
    }
    int[] res = new int[st.size()];
    for (int i = res.length - 1; i >= 0; i--) res[i] = st.pop();
    return res;
}
```

LC 394 · Decode String

MEDIUM

Pattern: Nested Dual Stack**Key Insight:** Push buffer on `[`, repeat on `.`.

```
public String decodeString(String s) {
    Deque<Integer> countSt = new ArrayDeque<>();
    Deque<StringBuilder> strSt = new ArrayDeque<>();
    StringBuilder curr = new StringBuilder(); int k = 0;
    for (char c : s.toCharArray()) {
        if (Character.isDigit(c)) k = k * 10 + (c - '0');
        else if (c == '[') {
            countSt.push(k); strSt.push(curr); curr = new StringBuilder(); k = 0;
        } else if (c == ']') {
            StringBuilder prev = strSt.pop(); int cnt = countSt.pop();
            for (int i = 0; i < cnt; i++) prev.append(curr);
            curr = prev;
        } else curr.append(c);
    }
    return curr.toString();
}
```

LC 84 · Largest Rectangle in Histogram

HARD

Pattern: Monotonic Increasing**Key Insight:** Width $= i - \text{stack.peek()} - 1$.

```
public int largestRectangleArea(int[] heights) {
    int n = heights.length, maxArea = 0;
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i <= n; i++) {
        int h = (i == n) ? 0 : heights[i];
        while (!st.isEmpty() && h < heights[st.peek()]) {
            int height = heights[st.pop()];
            int width = st.isEmpty() ? i : i - st.peek() - 1;
            maxArea = Math.max(maxArea, height * width);
        }
        st.push(i);
    }
}
```

LC 22 · Generate Parentheses

MEDIUM

Pattern: Backtracking Stack**Key Insight:** Add `(` if open < N, add `)` if close < open.

```
public List<String> generateParenthesis(int n) {
    List<String> res = new ArrayList<>();
    backtrack(res, new StringBuilder(), 0, 0, n);
    return res;
}

private void backtrack(List<String> res, StringBuilder sb, int open, int close, int n) {
    if (sb.length() == 2 * n) { res.add(sb.toString()); return; }
    if (open < n) { sb.append('('); backtrack(res, sb, open + 1, close, n); sb.deleteCharAt(sb.length() - 1); }
    if (close < open) { sb.append(')'); backtrack(res, sb, open, close + 1, n); }
}
```

```
}  
return maxArea;  
}
```

```
n); sb.deleteCharAt(sb.length() - 1);  
}  
}
```

🔍 DRY RUN — Daily Temperatures ([73, 74, 75, 71, 69, 72])

Index	Temp	Stack State (Indices)	Action / Result Assignment
0	73	[0]	Push 0
1	74	[1]	74 > 73 -> Pop 0! `res[0] = 1 - 0 = 1`. Push 1
2	75	[2]	75 > 74 -> Pop 1! `res[1] = 2 - 1 = 1`. Push 2
3	71	[2, 3]	71 < 75 -> Push 3
4	69	[2, 3, 4]	69 < 71 -> Push 4
5	72	[2, 5]	72 > 69 -> Pop 4! `res[4]=1`. 72 > 71 -> Pop 3! `res[3]=2`. Push 5

📐 MATHEMATICAL PROOF — O(N) MONOTONIC STACK

Claim: The inner `while (!stack.isEmpty() && ...)` loop does not make Monotonic Stack $O(N^2)$.

Proof: Each element is pushed onto the stack exactly once. An element can be popped from the stack at most once. Total pops across entire execution $\leq N$. Total operations $\$= N \text{ \texttt{pushes}} + \text{at most } N \text{ \texttt{pops}} = 2N \rightarrow \$ \textbf{Strictly } O(N) \textbf{ Time!}$

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Use `ArrayDeque` instead of `java.util.Stack`
- ☐ Code Valid Parentheses (LC 20) in 2m
- ☐ Implement Min Stack $O(1)$ operations
- ☐ Code Daily Temperatures with Monotonic Stack
- ☐ Write RPN Evaluator (LC 150)
- ☐ Code Car Fleet sorting by position desc
- ☐ Code Largest Rectangle in Histogram (LC 84)
- ☐ Prove why Monotonic Stack is $O(N)$

👛 GOLDEN RULES — NEVER FORGET

- Rule 1 — Store Indices, Not Values**

In Monotonic Stack problems calculating distances (e.g. Daily Temperatures, Histogram width), store ARRAY INDICES in the Stack!
- Rule 2 — Sentinel 0 Height**

In Largest Rectangle in Histogram, append a dummy 0 height at index N to force popping all remaining elements from the stack!